



acm International Collegiate
Programming Contest

IBM

event
sponsor

字符串

Ciallo~(∠・ω<)〃☆

目 录

字符串	2
字符串哈希	2
双哈希	4
KMP	6
前缀统计	8
循环节	9
最长公共子串	10
exKMP	12
Trie字典树	13
字符串统计	13
前缀统计	15
最大异或对	18
最小异或对	22
AC自动机	22
回文串	27
Manacher	28
最小表示法	29
后缀数组	30
排序+哈希+二分	30
倍增实现	32
height数组	33
其它	34

字符串

`string s = "ABCDE"`

子串(substring): $s[i] \sim s[j]$ 连续的一段,如:BCD

子序列(subsequence): $s[i] \sim s[j]$ 将若干元素提取出来并不改变相对位置形成的序列, 如:ACD

前缀 $s[0 \sim i]$: A、AB、ABC、ABCD、ABCDE

真前缀: 不包括本身的前缀 A、AB、ABC、ABCD

后缀 $s[i \sim n-1]$: E、DE、CDE、BCDE、ABCDE

真后缀: 不包括本身的后缀 E、DE、CDE、BCDE

回文串: 是正着写和倒着写相同的字符串, 如aba、aa、abcba

字典序: 以第 i 个字符作为第 i 关键字进行大小比较, 空字符小于字符集内任何字符

如: $abc < adc$ 、 $a < aa$ 、 $abcd < adcd$

border: 字符串中真前缀和真后缀相等的一段:

如 $f[a] = 0$ 、 $f[aba] = a$ 、 $f[abab] = ab$ 、 $f[ababa] = a, aba$ 、 $f[aaaa] = a, aa, aaa$

字符串哈希

[字符串哈希 - OI Wiki \(oi-wiki.org\)](https://oi-wiki.org/string/hash/)

只能匹配子串, 不能匹配子序列

全称字符串前缀哈希法,把字符串变成一个p进制数字(哈希值), 实现不同的字符串映射到不同的数字。

对形如 $X_1X_2X_3 \cdots X_{n-1}X_n$ 的字符串, 采用字符的ascii码乘上 P 的次方来计算哈希值。

映射公式(这里采用逆序): $(X_1 \times P^{n-1} + X_2 \times P^{n-2} + \dots + X_{n-1} \times P^1 + X_n \times P^0) \bmod Q$

$$h(S) := \left(\sum_{i=0}^{n-1} P^{n-i-1} * s[i] \right) \bmod Q$$

注意点:

- 1.任意字符不可以映射成0, 否则会出现不同的字符串都映射成0的情况, 比如A, AA, AAA皆为0
- 2.冲突问题: 通过巧妙设置底数P(131或13331), 模数 $[Q(2^{64})][ull, \text{自然溢出}]$ 的值, 减少冲突

问题是比较不同区间的子串是否相同, 就转化为对应的哈希值是否相同。

求一个字符串的哈希值就相当于求前缀和, 求一个字符串的子串哈希值就相当于求部分和。

前缀和公式 $h[i+1] = h[i] \times P + s[i]$, $0 \leq i \leq n-1$, h 为前缀和数组, s 为字符串数组

区间和公式 $h[l, r] = h[r+1] - h[l] \times P^{r-l+1}$

区间和公式的理解: ABCDE 与 ABC 的前三个字符值是一样, 只差两位, 乘上 P^2 把 ABC 变为 ABC00, 再用 ABCDE-ABC00得到 DE 的哈希值。


```

1  #include <iostream>
2  #include <set>
3  #include <vector>
4
5  namespace Hash{ //0_idx
6      const unsigned long long base = 131,mod = 1e9+7;
7      std::vector<unsigned long long>p(1,1),p2(1,1);//p[] 建议开定长数组提前预处理
8
9      template<typename T>
10     struct hx{
11         std::vector<unsigned long long>h;
12
13         hx(){}
14         hx(const T &s){
15             init(s.size());
16             h = std::vector<unsigned long long>(s.size()+1);
17             for(int i = 0;i < s.size();i++){
18                 h[i+1] = h[i] * base + s[i];
19             }
20         }
21         unsigned long long query(int l,int r){
22             return h[r+1] - h[l]*p[r-l+1];
23         }
24
25         void init(int id){
26             while(p.size() <= id){
27                 p.emplace_back(p.back()*base);
28             }
29         }
30     };
31
32     template<typename T>
33     struct hx2{ //双哈希
34         std::vector<unsigned long long>h,h2;
35
36         hx2(){}
37         hx2(const T &s){
38             int n = s.size();
39             init(s.size());
40             h.resize(n+1);h2.resize(n+1);
41             for(int i = 0;i < s.size();i++){
42                 h[i+1] = h[i]*base + s[i];
43                 h2[i+1] = (h2[i]*base + s[i]) % mod;
44             }
45         }
46
47         std::pair<unsigned long long,unsigned long long> query(int l,int r){
48             unsigned long long k1 = h[r+1] - h[l]*p[r-l+1];
49             unsigned long long k2 = (h2[r+1] - h2[l]*p2[r-l+1]%mod + mod) % mod;
50             return std::pair{k1,k2};
51         }
52     };

```

```

53     void init(int id){
54         while(p.size() <= id){
55             p.emplace_back(p.back()*base);
56             p2.emplace_back(p2.back()*base%mod);
57         }
58     }
59 };
60 };
61 using Hash::hx,Hash::hx2;
62
63 int main(){
64     std::ios::sync_with_stdio(false); std::cin.tie(0);
65     int n,q; std::cin >> n >> q;
66     std::string s; std::cin >> s;
67     s = ' ' + s;
68     hx hs(s);
69     while(q--){
70         int l1,r1,l2,r2; std::cin >> l1 >> r1 >> l2 >> r2;
71         if(hs.query(l1,r1) == hs.query(l2,r2)) {
72             std::cout << "Yes\n";
73         }
74         else std::cout << "No\n";
75     }
76 }

```

```

1 //简单求整个字符串的哈希值
2 const ull mod = 212370440130137957;
3 ull hs(string &s){ // 0_idx
4     ull ans = 0;
5     for(auto &x:s){
6         ans = (ans*131+x)%mod;
7     }
8     return ans;
9 }

```

双哈希

单哈希如大模数哈希/自然溢出哈希仍然可能会被hack
使用两个模数进行哈希得出两个不同哈希值，减少冲突概率

```

1 //常用模数取值
2 1e9+7
3 ull自然溢出
4 212370440130137957
5 111111111111111111 //19个1
6 (1LL << 31) - 1
7 998244353

```

```

1 //字符串哈希 数据加强版 https://www.luogu.com.cn/problem/U461211
2 //诺TLE,则尽量开固定数组且预处理p[]和p2[], 减小常数
3 //query(l,r)返回子串s[l~r]的哈希值
4 namespace Hash{
5     const unsigned long long base = 131,mod = 1e9+7;
6     std::vector<unsigned long long>p(1,1),p2(1,1);
7
8     template<typename T>
9     struct hx{ //单哈希 hx hs(s);
10         std::vector<unsigned long long>h;
11
12         hx(){}
13         hx(const T &s){
14             init(s.size());
15             h = std::vector<unsigned long long>(s.size()+1);
16             for(int i = 0;i < s.size();i++){
17                 h[i+1] = h[i] * base + s[i];
18             }
19         }
20         unsigned long long query(int l,int r){
21             return h[r+1] - h[l]*p[r-l+1];
22         }
23
24         void init(int id){
25             while(p.size() <= id){
26                 p.emplace_back(p.back()*base);
27             }
28         }
29     };
30
31     template<typename T>
32     struct hx2{//双哈希 hx2 hs(s);
33         std::vector<unsigned long long>h,h2;
34
35         hx2(){}
36         hx2(const T &s){
37             int n = s.size();
38             init(s.size());
39             h.resize(n+1);h2.resize(n+1);
40             for(int i = 0;i < s.size();i++){
41                 h[i+1] = h[i]*base + s[i];
42                 h2[i+1] = (h2[i]*base + s[i]) % mod;
43             }

```

```

44     }
45
46     std::pair<unsigned long long,unsigned long long> query(int l,int r){
47         unsigned long long k1 = h[r+1] - h[l]*p[r-l+1];
48         unsigned long long k2 = (h2[r+1] - h2[l]*p2[r-l+1]%mod + mod) % mod;
49         return std::pair{k1,k2};
50     }
51
52     void init(int id){
53         while(p.size() <= id){
54             p.emplace_back(p.back()*base);
55             p2.emplace_back(p2.back()*base%mod);
56         }
57     }
58 };
59 };
60 using Hash::hx,Hash::hx2;

```

KMP

一个模式串匹配一个/多个文本串

能够在线性时间内判定字符串 A[1~N] 是否为字符串 B[1~M] 的子串，并求出字符串A在字符串B中各次出现的位置。能比字符串哈希更高效准确地处理这个问题，并且能提供额外的信息

next[i]数组求的是子串s[1~i]的最长border(真前缀==真后缀)

```

1 //下标从1开始
2 #include <iostream>
3 using namespace std;
4 const int N = 1000006;
5 int ne[N],f[N];
6
7 int main(){
8     string a,b;
9     int n,m;
10    cin >> n >> a >> m >> b;
11    a = ' ' + a;b = ' ' + b;
12
13    ne[0] = 0;
14    for(int i = 2,j = 0;i <= n;i++){//自身匹配, i从2开始
15        while(j > 0 && a[i] != a[j+1]) j = ne[j];
16        if(a[i] == a[j+1]) j++;
17        ne[i] = j;
18    }
19
20    for(int i = 1,j = 0;i <= m;i++){//对目标字符串匹配,求a在b的所有出现下标
21        while(j > 0 && b[i] != a[j+1]) j = ne[j];

```

```

22     if(b[i] == a[j+1]) j++;
23     f[i] = j; //f[i]表示文本串以b[i]结尾的子串的后缀和模式串的前缀的最长匹配长度
24     if(f[i] == n){ cout << i - n << ' '; j = ne[j];}
25 }
26 }

```

```

1  //https://vjudge.net/problem/HDU-1711
2  #include <iostream>
3  #include <vector>
4
5  template<typename T>
6  struct KMP{//l_idx
7      T a;
8      std::vector<int> ne;
9
10     KMP(){}
11     KMP(const T&v){ init(v); }
12
13     void init(const T&v){
14         a = v;
15         ne.assign(a.size(), {});
16         for(int i = 2, j = 0; i < a.size(); i++){
17             while(j > 0 && a[i] != a[j+1]) j = ne[j];
18             if(a[i] == a[j+1]) j++;
19             ne[i] = j;
20         }
21     }
22
23     std::vector<int> kmp(const T&b){
24         std::vector<int> ans;
25         for(int i = 1, j = 0; i < b.size(); i++){
26             while(j > 0 && b[i] != a[j+1]) j = ne[j];
27             if(b[i] == a[j+1]) j++;
28             //f[i] = j;
29             if(j == a.size()-1) {
30                 ans.push_back(i-a.size()+2), j = ne[j]; //诺此处令j=0,即为求不重叠的
匹配
31             }
32         }
33         return ans;
34     }
35 };
36
37
38 void sol(){
39     int n,m; std::cin >> n >> m;
40     std::vector<int> a(n+1), b(m+1);
41     for(int i = 1; i <= n; i++){ std::cin >> a[i]; }
42     for(int i = 1; i <= m; i++){ std::cin >> b[i]; }
43
44     KMP<std::vector<int>> kmp(b);

```

```

45
46     auto ans = kmp.kmp(a); //求a中出现模版串的所有下标
47     if(ans.size()) std::cout << ans[0] << '\n';
48     else std::cout << -1 << '\n';
49 }
50
51 int main(){
52     std::ios::sync_with_stdio(false);std::cin.tie(0);
53     int t; std::cin >> t;
54     while(t--) sol();
55 }

```

前缀统计

统计每个前缀的出现次数

string s = abab; f(s) = a* 2 + ab*2 + aba + abab = 6

```

1 //原题数据较弱 https://acm.hdu.edu.cn/showproblem.php?pid=3336
2 #include <iostream>
3 #include <vector>
4
5 template<typename T>
6 struct KMP{ //1_idx
7     T a;
8     std::vector<int> ne;
9
10    KMP(){}
11    KMP(const T&v){ init(v); }
12
13    void init(const T&v){
14        a = v;
15        ne.assign(a.size(),{});
16        for(int i = 2,j = 0;i < a.size();i++){
17            while(j > 0 && a[i] != a[j+1]) j = ne[j];
18            if(a[i] == a[j+1]) j++;
19            ne[i] = j;
20        }
21    }
22
23    std::vector<int> kmp(const T&b){
24        std::vector<int>ans;
25        for(int i = 1,j = 0;i < b.size();i++){
26            while(j > 0 && b[i] != a[j+1]) j = ne[j];
27            if(b[i] == a[j+1]) j++;
28            //f[i] = j;
29            if(j == a.size()-1) ans.push_back(i-a.size()+2), j = ne[j];
30        }
31        return ans;
32    }

```

```

33 };
34
35 const int mod = 10007;
36
37 void sol(){
38     int n; std::cin >> n;
39     std::string s; std::cin >> s;
40     s = ' ' + s;
41     KMP<std::string>t(s);
42
43     auto ne = t.ne;
44
45     std::vector<int>f(n+1);
46     for(int i = 1;i <= n;i++) {
47         f[i] = f[ne[i]] + 1;
48     }
49
50     int ans = 0;
51     for(int i = 1;i <= n;i++) {
52         ans = (ans + f[i]) % mod;
53     }
54     std::cout << ans << '\n';
55 }
56
57 int main(){
58     std::ios::sync_with_stdio(false); std::cin.tie(0);
59     int t; std::cin >> t;
60     while(t--) sol();
61 }

```

循环节

字符串的循环节

若字符串S可以由子串A循环而成，则子串A是S的周期，如S = “abababab”，则周期= ab 、 abab

如果 $(ne[i] > 0 \ \&\& \ i \% [i - ne[i]] == 0)$ ，则前缀 $s[1 \sim i]$ 的最小循环节 $k = i - ne[i]$

```

1  //https://www.acwing.com/problem/content/143/
2  //给定字符串s,求具有循环节的前缀长度i和其最小循环节对应的循环次数
3  #include <iostream>
4  using namespace std;
5  const int N = 1000006;
6  int ne[N];
7
8  int main(){
9      int n,idx = 0;;
10     while(cin >> n,n){

```

```

11     cout << "Test case #" << ++idx << endl;
12     string s; cin >> s;
13     s = ' ' + s;
14     for(int i = 2, j = 0; i <= n; i++){
15         while(j > 0 && s[i] != s[j+1]) j = ne[j];
16         if(s[i] == s[j+1]) j++;
17         ne[i] = j;
18     }
19
20     for(int i = 1; i <= n; i++){
21         if(ne[i] && i%(i-ne[i]) == 0){
22             cout << i << ' ' << i/(i-ne[i]) << endl; //子串s[1~i]的最短循环节循
环的次数
23         }
24     }
25     cout << endl;
26 }
27 }

```

最长公共子串

求多个字符串的最长公共子串

二分+KMP

```

1 //https://vjudge.net/problem/HDU-2328
2 //求长公共子串, 若不唯一, 输出字典序最小的
3 #include <iostream>
4 #include <algorithm>
5 #include <vector>
6
7 template<typename T>
8 struct KMP{ //l_idx
9     T a;
10    std::vector<int> ne;
11
12    KMP(){}
13    KMP(const T&v){ init(v); }
14
15    void init(const T&v){
16        a = v;
17        ne.assign(a.size(), {});
18        for(int i = 2, j = 0; i < a.size(); i++){
19            while(j > 0 && a[i] != a[j+1]) j = ne[j];
20            if(a[i] == a[j+1]) j++;
21            ne[i] = j;
22        }
23    }
24
25    std::vector<int> kmp(const T&b){

```



```

26     std::vector<int>ans;
27     for(int i = 1,j = 0;i < b.size();i++){
28         while(j > 0 && b[i] != a[j+1]) j = ne[j];
29         if(b[i] == a[j+1]) j++;
30         //f[i] = j;
31         if(j == a.size()-1) {
32             ans.push_back(i-a.size()+2);
33             j = ne[j];
34             return ans;//匹配成功一次直接返回即可
35         }
36     }
37     return ans;
38 }
39 };
40
41 const int N = 4003;
42 int n;
43 std::string s[N];
44 std::string ans;
45
46 bool check(int len){
47     bool ok = 0;
48     for(int i = 1;i+len-1 < s[1].size();i++){
49         auto now = s[1].substr(i,len);
50         if(ok && (ans <= now)) continue;//剪枝
51
52         KMP<std::string>t(' ' + now);
53         bool flag = 1;
54         for(int j = 2;j <= n;j++){
55             if(t.kmp(s[j]).empty()) {
56                 flag = 0;
57                 break;
58             }
59         }
60         if(flag) {
61             ok = 1;
62             if(ans.size() < now.size()) ans = now;
63             else if(ans.size() == now.size()) ans = std::min(ans,now);
64         }
65     }
66     return ok;
67 }
68
69 void sol(){
70     ans = "";
71     int mn = N,p = 1;
72     for(int i = 1;i <= n;i++) {
73         std::cin >> s[i];
74         if(s[i].size() < mn) {
75             mn = s[i].size();
76             p = i;
77         }

```

```

78     s[i] = ' ' + s[i];
79 }
80 std::swap(s[l],s[p]);
81 int l = 0,r = mn;
82 while(l < r){
83     int mid = l + r + 1 >> 1;
84     if(check(mid)) l = mid;
85     else r = mid - 1;
86 }
87 if(ans.size()) std::cout << ans << '\n';
88 else std::cout << "IDENTITY LOST\n";
89 }
90
91 int main(){
92     std::ios::sync_with_stdio(false); std::cin.tie(0);
93     while(std::cin >> n,n) sol();
94 }

```

exKMP

也被称之为 *Z Algorithm* 或扩展 *KMP*。

对于一个长度为 n 的字符串 $s(1_idx)$, 定义 $z[i]$ 表示 s 和 $s[i,n]$ (即以 $s[i]$ 开头的后缀)的最长公共前缀, 我们将 z 称之为 s 的 z 函数。考虑到 $z[1]$ 的特殊性质, 一般将其设置为 $z[1]=0$ 。例如 $z("aaaba") = \{0,2,1,0,1\}$ 。

时间复杂度 $O(N)$

```

1  template<typename T>
2  struct EXKMP{ //1_idx
3      T a;
4      std::vector<int>z;
5
6      EXKMP(){}
7      EXKMP(const T&v){ init(v); }
8
9      void init(const T &v) {
10         a = v;
11         z = std::vector<int>(a.size());
12         for(int i = 2,k = 1;i < a.size();i++){
13             if(k+z[k]-i <= z[i-k+1]) {
14                 z[i] = k+z[k]-i;
15                 if(z[i] < 0) z[i] = 0;
16                 while(i+z[i] < a.size() && a[z[i]+1] == a[z[i]+i]) ++z[i];
17                 k = i;
18             }
19             else{
20                 z[i] = z[i-k+1];

```

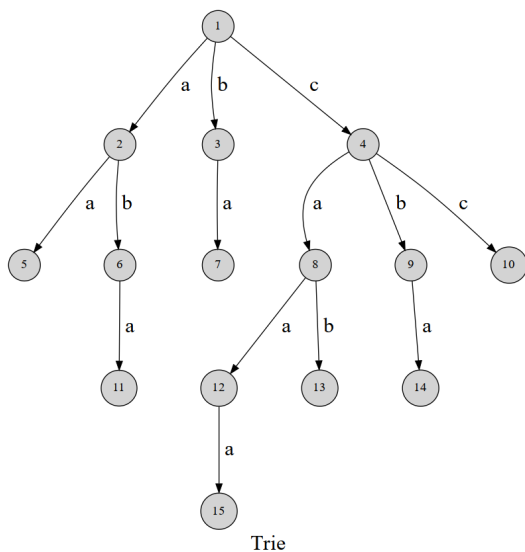
```

21     }
22     }
23     z[1] = 0; //or a.size()-1?
24 }
25
26 std::vector<int> exkmp(const T&v){//ans[i]表示v的后缀子串v.substr(i)与字符串a的
    最长公共前缀
27     std::vector<int>ans(v.size());
28     for(int i = 1,k = 1;i < v.size();i++){
29         if(k+ans[k]-i <= z[i-k+1]) {
30             ans[i] = k+ans[k]-i;
31             if(ans[i] < 0) ans[i] = 0;
32             while(i+ans[i] < v.size() && ans[i] < a.size() && a[ans[i]+1] ==
v[ans[i]+i]) ++ans[i];
33             k = i;
34         }
35         else{
36             ans[i] = z[i-k+1];
37         }
38     }
39     return ans;
40 }
41 };

```

Trie字典树

高效地存储和查找字符串集合的数据结构



字符串统计

```

1 #include <iostream>
2 #include <cstring>
3
4 namespace TRIE{//0_idx

```

```

5     const int N = 100005,M = 26;//max_s.length
6     int son[N][M],cnt[N],idx;
7
8     struct Trie{
9
10        Trie(){idx = 0;init(idx);}
11
12        void init(int p){
13            cnt[p] = 0;
14            memset(son[p], 0, sizeof(son[p]));
15        }
16
17        int get(char x){//
18            if(x >= 'a' && x <= 'z') return x - 'a';
19            if(x >= 'A' && x <= 'Z') return x - 'A' + 26;
20            if(x >= '0' && x <= '9') return x - '0' + 52;
21            return -1;
22        }
23
24        void insert(const std::string &s){
25            int p = 0;
26            for(int i = 0;i < s.size();i++){
27                int u = get(s[i]);
28                if(!son[p][u]) {
29                    son[p][u] = ++idx;
30                    init(idx);
31                }
32                p = son[p][u];
33                //cnt[p]++; //前缀++
34            }
35            cnt[p]++; //字符串++
36        }
37
38        int query(const std::string &s){
39            int p = 0;
40            for(int i = 0;i < s.size();i++){
41                int u = get(s[i]);
42                if(!son[p][u]) return 0;
43                p = son[p][u];
44            }
45            return cnt[p];
46        }
47    };
48 };
49 using TRIE::Trie;
50
51
52 int main(){
53     Trie t;
54     int q; std::cin >> q;
55     while(q--){
56         char op; std::string s;

```

```

57     std::cin >> op >> s;
58     if(op == 'I') { t.insert(s); }
59     else { std::cout << t.query(s) << '\n'; }
60 }
61 }

```

前缀统计

给定 N 个字符串 $S_1, S_2 \dots S_N$ ，接下来进行 M 次询问，每次询问给定一个字符串 T ，求 $S_1 \sim S_N$ 中多少个字符串是 T 的前缀。

```

1  //https://www.acwing.com/problem/content/144/
2  #include <iostream>
3  #include <cstring>
4
5  namespace TRIE{//0_idx
6      const int N = 1000006,M = 26;//max_s.length
7      int son[N][M],cnt[N],idx;
8
9      struct Trie{
10
11          Trie(){init();}
12
13          void init(){
14              idx = cnt[0] = 0;
15              memset(son[0], 0, sizeof(son[0]));
16          }
17
18          int get(char x){
19              if(x >= 'a' && x <= 'z') return x - 'a';
20              if(x >= 'A' && x <= 'Z') return x - 'A' + 26;
21              if(x >= '0' && x <= '9') return x - '0' + 52;
22              return 0;
23          }
24
25          void insert(const std::string &s){
26              int p = 0;
27              for(int i = 0;i < s.size();i++){
28                  int u = get(s[i]);
29                  if(!son[p][u]) {
30                      son[p][u] = ++idx;
31                      cnt[idx] = 0;
32                      std::memset(son[idx],0,sizeof son[idx]);
33                  }
34                  p = son[p][u];
35                  //cnt[p]++; //suff++
36              }
37              cnt[p]++; //string++;
38          }

```

```

39
40     int query(const std::string &s){
41         int ans = 0;
42         int p = 0;
43         for(int i = 0;i < s.size();i++){
44             int u = get(s[i]);
45             if(!son[p][u]) return ans;
46             p = son[p][u];
47             ans += cnt[p];
48         }
49         // return cnt[p];
50         return ans;
51     }
52 };
53 };
54 using TRIE::Trie;
55
56 int main(){
57     Trie t;
58     int n,q; std::cin >> n >> q;
59     while(n--){
60         std::string s; std::cin >> s;
61         t.insert(s);
62     }
63     while(q--){
64         std::string s; std::cin >> s;
65         std::cout << t.query(s) << '\n';
66     }
67 }

```

给定 N 个字符串 $S_1, S_2 \dots S_N$ ，接下来进行 M 次询问，每次询问给定一个字符串 T ，求 $S_1 \sim S_N$ 中 T 是多少个字符串的前缀

```

1 //https://www.luogu.com.cn/problem/P8306
2 #include <iostream>
3 #include <cstring>
4
5 namespace TRIE{//0_idx
6     const int N = 3000006,M= 70;//max_s.length
7     int son[N][M],cnt[N],idx;
8
9     struct Trie{
10
11         Trie(){init();}
12
13         void init(){
14             cnt[0] = 0;
15             memset(son[0], 0, sizeof(son[0]));
16             idx = 0;
17         }

```

```

18
19     int get(char x){
20         if(x >= 'a' && x <= 'z') return x - 'a';
21         if(x >= 'A' && x <= 'Z') return x - 'A' + 26;
22         if(x >= '0' && x <= '9') return x - '0' + 52;
23         return 0;
24     }
25
26     void insert(const std::string &s){
27         int p = 0;
28         for(int i = 0; i < s.size(); i++){
29             int u = get(s[i]);
30             if(!son[p][u]) {
31                 son[p][u] = ++idx;
32                 cnt[idx] = 0;
33                 std::memset(son[idx], 0, sizeof son[idx]);
34             }
35             p = son[p][u];
36             cnt[p]++;
37         }
38     }
39
40     int query(const std::string &s){
41         int p = 0;
42         for(int i = 0; i < s.size(); i++){
43             int u = get(s[i]);
44             if(!son[p][u]) return 0;
45             p = son[p][u];
46         }
47         return cnt[p];
48     }
49 };
50
51 using TRIE::Trie;
52
53 void sol(){
54     Trie t;
55     int n, q; std::cin >> n >> q;
56     while(n--){
57         std::string s; std::cin >> s;
58         t.insert(s);
59     }
60     while(q--){
61         std::string s; std::cin >> s;
62         std::cout << t.query(s) << '\n';
63     }
64 }
65
66 int main(){
67     std::ios::sync_with_stdio(false); std::cin.tie(0);
68     int t; std::cin >> t;
69     while(t--) sol();

```

最大异或对

在给定的 N 个整数 $A_1, A_2, \dots, A_N$ 中选出两个进行 xor (异或) 运算, 得到的结果最大是多少?

```

1  //https://www.luogu.com.cn/problem/P10471
2  #include <iostream>
3  #include <cstring>
4
5  namespace TRIE{//0_idx
6      const int N = 100005*32;
7      int son[N][2],cnt[N],idx;
8
9      struct Trie{
10
11          Trie(){init();}
12
13          void init(){
14              idx = cnt[0] = 0;
15              memset(son[0], 0, sizeof(son[0]));
16          }
17
18          int get(char x){
19              if(x >= 'a' && x <= 'z') return x - 'a';
20              if(x >= 'A' && x <= 'Z') return x - 'A' + 26;
21              if(x >= '0' && x <= '9') return x - '0' + 52;
22              return 0;
23          }
24
25          void insert(const int &x){
26              int p = 0;
27              for(int i = 30;i >= 0;i--){
28                  bool u = x >> i & 1;
29                  if(!son[p][u]) {
30                      son[p][u] = ++idx;
31                      cnt[idx] = 0;
32                      std::memset(son[idx],0,sizeof son[idx]);
33                  }
34                  p = son[p][u];
35              }
36              cnt[p]++;
37          }
38
39          int query(const int &x){
40              int ans = 0;
41              int p = 0;
42              for(int i = 30;i >= 0;i--){

```



```

43         bool u = x >> i & 1;
44         if(son[p][!u]) { //高位开始，每次找与当前位相反的
45             ans = ans<<1|1;
46             p = son[p][!u];
47         }
48         else{
49             ans = ans<<1;
50             p = son[p][u];
51         }
52     }
53     return ans;
54 }
55 };
56
57 using TRIE::Trie;
58
59 int main(){
60     Trie t;
61     int n; std::cin >> n;
62     int ans = 0;
63     while(n--){
64         int x; std::cin >> x;
65         ans = std::max(ans,t.query(x));
66         t.insert(x);
67     }
68     std::cout << ans;
69 }

```

```

1  //xor_Trie 封装
2  //插入/删除一个数
3  //求x与另一个数异或的最大/最小值
4  namespace TRIE{ //0_idx x<(2^31)
5      const int SIZ = 100005*32;
6      int son[SIZ][2],cnt[SIZ],idx;
7
8      struct xor_Trie{
9
10         xor_Trie(){
11             idx = 0;
12             init(idx);
13         }
14
15         void init(int p){
16             cnt[p] = 0;
17             memset(son[p], 0, sizeof(son[p]));
18         }
19
20         void insert(const int &x){
21             int p = 0;

```

```

22         for(int i = 30; i >= 0; i--){
23             bool u = x >> i & 1;
24             if(!son[p][u]) {
25                 son[p][u] = ++idx;
26                 init(idx);
27             }
28             p = son[p][u];
29             cnt[p]++;
30         }
31     }
32
33     void erase(const int &x){
34         int p = 0;
35         for (int i = 30; i >= 0; i--) {
36             bool u = x >> i & 1;
37             p = son[p][u];
38             cnt[p]--;
39         }
40     }
41
42     int findMaxXor(const int &x){
43         int ans = 0;
44         int p = 0;
45         for(int i = 30; i >= 0; i--){
46             bool u = x >> i & 1;
47             if(son[p][!u] && cnt[son[p][!u]]) {
48                 ans |= 1 << i;
49                 p = son[p][!u];
50             }
51             else{
52                 p = son[p][u];
53             }
54         }
55         return ans;
56     }
57
58     int findMinXor(const int &x){
59         int ans = 0;
60         int p = 0;
61         for (int i = 30; i >= 0; i--) {
62             bool u = x >> i & 1;
63             if (son[p][u] && cnt[son[p][u]]) {
64                 p = son[p][u];
65             }
66             else {
67                 ans |= 1 << i;
68                 p = son[p][!u];
69             }
70         }
71         return ans;
72     }
73 };

```

```

74 };
75 using TRIE::xor_Trie;

```

P4551 最长异或路径

给定一棵 n 个点的带边权树，求树中最大的异或路径值。

思路：设 $d[x]$ 表示根节点到 x 的路径异或值，则有 $d[x] = d[\text{father}[x]] \oplus \text{weight}(x, \text{father}[x])$ ，DFS 一遍可以求出所有 $d[i]$ ， x 到 y 的路径异或值就等于 $d[x] \oplus d[y]$ (x 到根 和 y 到根，重叠部分异或抵消了，就相当于 x 到 y)，问题就变为 $d[N]$ 中求最大异或对

```

1  #include <iostream>
2  #include <cstring>
3  using namespace std;
4  const int N = 200005;
5  int n;
6  int a[N];
7  int h[N], ne[N], e[N], w[N], idx;
8  int son[N*32][2], tot;
9  int d[N];
10
11 void add(int a, int b, int c){
12     w[idx] = c, e[idx] = b, ne[idx] = h[a], h[a] = idx++;
13 }
14
15 void dfs(int u, int fa){
16     for(int i = h[u]; ~i; i = ne[i]){
17         int k = e[i];
18         if(k != fa){
19             d[k] = d[u] ^ w[i];
20             dfs(k, u);
21         }
22     }
23 }
24
25 void insert(int x){
26     int p = 0;
27     for(int i = 31; i >= 0; i--){
28         bool u = x >> i & 1;
29         if(!son[p][u]) son[p][u] = ++tot;
30         p = son[p][u];
31     }
32 }
33
34 int query(int x){
35     int p = 0;
36     int ans = 0;
37     for(int i = 31; i >= 0; i--){
38         bool u = x >> i & 1;
39         if(son[p][!u]) {

```

```

40         ans = ans*2 + 1;
41         p = son[p][!u];
42     }
43     else{
44         ans = ans*2;
45         p = son[p][u];
46     }
47 }
48 return ans;
49 }
50
51 int main(){
52     memset(h,-1,sizeof h);
53     cin >> n;
54     for(int i = 1;i < n;i++){
55         int a,b,c;cin >> a >> b >> c;
56         add(a,b,c);
57         add(b,a,c);
58     }
59
60     dfs(1,1);
61
62     int ans = 0;
63     for(int i = 1;i <= n;i++){
64         ans = max(ans,query(d[i]));
65         insert(d[i]);
66     }
67     cout << ans;
68 }

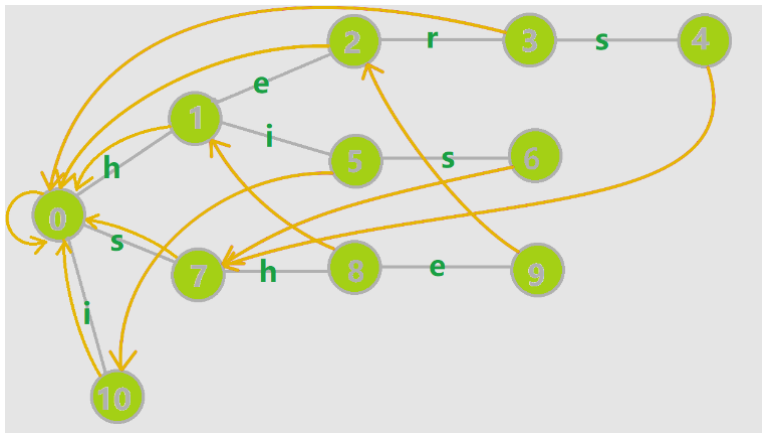
```

最小异或对

将数组排序后，相邻元素异或值最小的一对即为最小异或对。 [G - Minimum Xor Pair Query](#).

AC自动机

多个模式串匹配一个/多个文本串



- 构建Trie前缀树
- 构建失配指针：状态 u 的 fail 指针指向另一个状态 v ，其中 $v \in Trie$ ，且 v 是 u 的最长后缀（即在若干个后缀状态中取最长的一个作为 fail 指针）。
- 查询文本串：例如，当前要匹配的文本串为 shers，在字典树上找到状态 9:she 时，此时无后续节点，失配，直接跳转到状态 9:she 的最长后缀状态 2:he，然后继续匹配。

```

1 //模版题 AC自动机(简单版) https://www.luogu.com.cn/problem/P3808
2 //给定n个模式串和一个文本串,求文本串中出现了多少个模式串
3 #include <iostream>
4 #include <queue>
5 #include <cstring>
6
7 namespace ACAM{//0_idx
8     const int N = 1000006, M = 26;//N:所有模式串的总长度
9     int son[N][M],cnt[N],fail[N],idx;
10
11     struct Trie{
12
13         Trie(){idx = 0;init(idx);}
14
15         void init(int p){
16             fail[p] = cnt[p] = 0;
17             std::memset(son[p], 0, sizeof(son[p]));
18         }
19
20         int get(char x){
21             if(x >= 'a' && x <= 'z') return x - 'a';
22             if(x >= 'A' && x <= 'Z') return x - 'A' + 26;
23             if(x >= '0' && x <= '9') return x - '0' + 52;
24             return -1;
25         }
26
27         void insert(const std::string &s){
28             int p = 0;
29             for(int i = 0;i < s.size();i++){
30                 int u = get(s[i]);
31                 if(!son[p][u]) {
32                     son[p][u] = ++idx;

```

```

33         init(idx);
34     }
35     p = son[p][u];
36 }
37 cnt[p]++;
38 }
39
40 void get_fail() { // 构建失配指针
41     fail[0] = 0;
42     std::queue<int> q;
43     for (int i = 0; i < M; i++) { // 第二层的 fail 全部指向根节点
44         if (son[0][i]) {
45             fail[son[0][i]] = 0;
46             q.push(son[0][i]);
47         }
48     }
49     while (q.size()) {
50         int p = q.front();
51         q.pop();
52         for (int u = 0; u < M; u++) {
53             if (son[p][u]) { // 如果子节点存在
54                 fail[son[p][u]] = son[fail[p]][u]; // 子节点的 fail 指针指向当前节点的 fail 指针指向的相同子节点
55                 q.push(son[p][u]); // 子节点入队
56             }
57             else { // 如果子节点不存在
58                 son[p][u] = son[fail[p]][u]; // 直接让子节点指向当前节点的 fail 指针指向的相同子节点
59             }
60         }
61     }
62 }
63
64 int query(const std::string &s) {
65     int p = 0, ans = 0;
66     for (int i = 0; i < s.size(); i++) {
67         int u = get(s[i]);
68         p = son[p][u]; // 依次读入单词, 然后指针跳转到子节点
69         ans += cnt[p];
70         cnt[p] = 0;
71         // 统计答案
72         //         ans += cnt[t];
73         //         cnt[t] = -1; // 清空, 本题出现多次只算一次, 诸多文本串匹配, 可以开个 vis 数组标记状态 t
74         //     }
75     }
76     return ans;
77 }
78 };
79 };
80 using ACAM::Trie; // Trie t;

```

```

81
82 int main(){
83     Trie t;
84     int n; std::cin >> n;
85     std::string s;
86     for(int i = 1; i <= n; i++){
87         std::cin >> s;
88         t.insert(s);
89     }
90     t.get_fail();
91     std::cin >> s;
92     std::cout << t.query(s) << '\n';
93 }

```

拓扑排序优化

fail 指针的一个性质：一个 AC 自动机中，如果只保留 fail 边，那么剩余的图一定是一棵树。这样 AC 自动机的匹配就可以转化为在 fail 树上的链求和问题

观察到时间主要浪费在在每次都要跳 fail。如果我们可以预先记录，最后一并求和，那么效率就会优化。

于是我们按照 fail 树，做一次内向树上的拓扑排序，就能一次性求出所有模式串的出现次数。

```

1 //https://www.luogu.com.cn/problem/P5357
2 //给定n个模式串和一个文本串,分别求出每个模式串在文本串中出现的次数(可重叠)
3 #include <bits/stdc++.h>
4 using namespace std;
5
6 int id;
7 std::vector<int>vis,mp;
8
9 namespace ACAM{//0_idx
10     const int N = 200005, M = 26;
11     int son[N][M],cnt[N],fail[N],idx;
12     int du[N],ans[N];
13
14     struct Trie{
15
16         Trie(){idx = 0;init(idx);}
17
18         void init(int p){
19             ans[p] = du[p] = 0;
20             fail[p] = cnt[p] = 0;
21             std::memset(son[p], 0, sizeof(son[p]));
22         }
23
24         int get(char x){

```

```

25     if(x >= 'a' && x <= 'z') return x - 'a';
26     if(x >= 'A' && x <= 'Z') return x - 'A' + 26;
27     if(x >= '0' && x <= '9') return x - '0' + 52;
28     return -1;
29 }
30
31 void insert(const std::string &s){
32     int p = 0;
33     for(int i = 0; i < s.size(); i++){
34         int u = get(s[i]);
35         if(!son[p][u]) {
36             son[p][u] = ++idx;
37             init(idx);
38         }
39         p = son[p][u];
40     }
41     if(!cnt[p]) cnt[p] = id; //状态p对应字符串id+去重
42     mp[id] = cnt[p]; //字符串id对应状态p的id
43 }
44
45 void get_fail(){
46     std::queue<int> q;
47     for(int i = 0; i < M; i++){
48         if(son[0][i]) {
49             fail[son[0][i]] = 0;
50             q.push(son[0][i]);
51         }
52     }
53     while(q.size()){
54         int p = q.front();
55         q.pop();
56         for(int u = 0; u < M; u++){
57             if(son[p][u]) {
58                 fail[son[p][u]] = son[fail[p]][u];
59                 du[son[fail[p]][u]]++; //额外记录入度
60                 q.push(son[p][u]);
61             }
62             else {
63                 son[p][u] = son[fail[p]][u];
64             }
65         }
66     }
67 }
68
69 int query(const std::string &s){
70     int p = 0;
71     for(int i = 0; i < s.size(); i++){
72         int u = get(s[i]);
73         p = son[p][u];
74         ans[p]++;
75     }
76     topu();

```



```

77         return 0;
78     }
79     void topu(){//拓扑排序统计答案
80         std::queue<int>q;
81         for(int i = 1;i <= idx;i++){
82             if(!du[i]) q.push(i);//入度为零的点，必定是一个 Fail 链的末尾
83         }
84         while(q.size()){
85             int p = q.front();
86             q.pop();
87             vis[cnt[p]] = ans[p];
88
89             ans[fail[p]] += ans[p];
90             if(!--du[fail[p]]) q.push(fail[p]);
91         }
92     }
93 };
94 };
95 using ACAM::Trie; //Trie t;
96
97 const int N = 200005;
98 std::string s[N];
99
100 int main(){
101     Trie t;
102     int n; std::cin >> n;
103     vis = mp = std::vector<int>(n+1,0);
104
105     for(int i = 1;i <= n;i++){
106         std::cin >> s[i];
107         id = i;
108         t.insert(s[i]);
109     }
110     t.get_fail();
111
112     std::cin >> s[0];
113     t.query(s[0]);
114
115     for(int i = 1;i <= n;i++){
116         std::cout << vis[mp[i]] << '\n';
117     }
118 }

```

回文串

Manacher

$O(N)$ 求得对于每个 i 的最长回文长度，这里下标从0开始

$t = "abbba"$ 则对应 $s = " \#a\#b\#b\#b\#a\#"$ ，可以同时处理奇偶回文串，而不必分开讨论，对 s 求得 $d1[]$ 为以 $s[i]$ 为中心的最长回文串长度。

对于 $t[i]$ 来说, $d1[i*2]-1$ 即为以 i 为中心，最长的奇数长度回文串

$d1[i*2+1]-1$ 即为以 $(i, i+1)$ 中点为中心最长的偶数长度回文串

```
1 //https://www.luogu.com.cn/problem/P3805
2 //模板题,给定字符串,求最长的回文子串长度
3 template<typename T>
4 struct Manacher{ //l_idx
5     int n,ans;
6     T s;
7     std::vector<int>d;
8
9     Manacher(){}
10    Manacher(const T &v){
11        init(v);
12    }
13
14    void init(const T &v){
15        s = " \#";
16        for(int i = 1;i < v.size();i++){
17            s += v[i];
18            s += '\#';
19        }
20        n = s.size()-1;
21        d = std::vector<int>(n+1,0);
22
23        ans = 0;
24        for(int i = 1,l = 0,r = 0;i <= n;i++){
25            if(i > r) d[i] = 1;
26            else d[i] = std::min(d[l*2-i],r-i+1);
27            while(i-d[i] >= 1 && i+d[i] <= n && s[i-d[i]] == s[i+d[i]]) d[i]++;
28            if(i+d[i]-1 > r) l = i,r = i+d[i]-1;
29            ans = std::max(ans,d[i]-1); //减去加入的'\#'
30        }
31    }
32
33    bool query(int l,int r){ //查询区间是否为回文串
34        l <= 1,r <= 1;
35        int mid = l + r >> 1;
36        return d[mid]-1 >= r-mid;
37    }
38 };
39
40 int main(){
41     std::string s;
42     std::cin >> s;
```

```

43
44     Manacher t(s);
45     std::cout << t.ans;
46 }

```

最小表示法

求s的循环同构中字典序最小，且下标最小的一个

时间复杂度 $O(N)$

```

1  //https://www.luogu.com.cn/problem/P1368
2  #include <iostream>
3  #include <vector>
4
5  template<typename T>
6  int get_min(T a){//1_idx
7      int n = a.size()-1;
8      for(int i = 1;i <= n;i++) a.push_back(a[i]);
9      int i = 1,j = 2;
10     while(i <= n && j <= n){
11         int k = 0;
12         while(k < n && a[i+k] == a[j+k]) k++;
13         if(k == n) break;
14         if(a[i+k] > a[j+k]){//诺将符号取反即为最大表示法
15             i += k+1;//诺a[i+k]>a[j+k],则a[i]~a[i+k]>a[j]
16             if(i == j) i++;//保持i != j
17         }
18         else{//a[j]同理
19             j += k+1;
20             if(i == j) j++;
21         }
22     }
23     return std::min(i,j);
24 }
25
26
27 int main(){
28     int n; std::cin >> n;
29     std::vector<int>a(n+1);
30     for(int i = 1;i <= n;i++){
31         scanf("%d",&a[i]);
32     }
33
34     int id = get_min(a);

```

```

35
36     for(int i = 0; i < n; i++){
37         printf("%d ", a[(id+i-1)%n+1]);
38     }
39 }

```

```

1 //https://acm.hdu.edu.cn/showproblem.php?pid=2609
2 //若两个字符串具有循环同构，则称两个字符串本质相同，求本质不同的字符串的有多少种
3 //思路：将每个字符串的最小表示法用set存起来，最后容器的大小即为答案

```

后缀数组

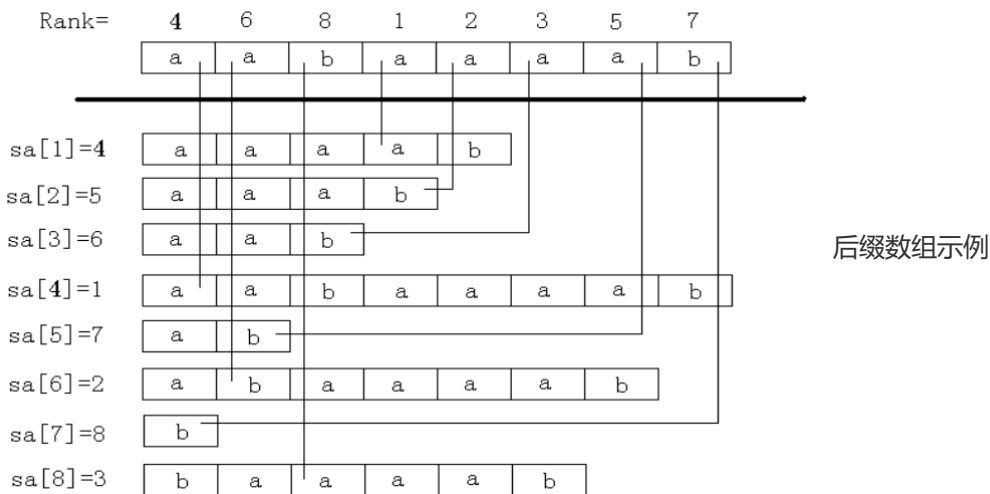
这里假定字符串长度为 n ，下标从1开始。“后缀”表示以第 i 个字符开头的后缀，储存时用 i 表示字符串 s 的后缀 $s[i...n]$ 。

后缀数组 $sa[i]$ 表示将所有后缀排序后，第 i 小的后缀的编号。

排名数组 $rk[i]$ 表示 后缀 i 的排名，重要的辅助数组。

这两个数组满足性质： $sa[rk[i]] = rk[sa[i]] = i$ 。

height数组 $height[i]$ ，即 $lcp(sa[i], sa[i-1])$



[P10469 后缀数组 - 洛谷 \(luogu.com.cn\)](https://www.luogu.com.cn/problem/P10469)

排序+哈希+二分

时间复杂度 $O(n \log^2 n)$

```

1 #include <iostream>
2 #include <algorithm>
3 using namespace std;
4 const unsigned long long base = 131;
5 const int N = 300005;

```

```

6  int n;
7  string s;
8  int sa[N],rk[N];
9  unsigned long long hs[N],p[N];
10
11 void init(string &s){
12     hs[0] = 0;
13     p[0] = 1;
14     for(int i = 0;i < s.size();i++){
15         p[i+1] = p[i] * base;
16         hs[i+1] = hs[i] * base + s[i];
17     }
18 }
19
20 unsigned long long query(int l,int r){
21     return hs[r] - hs[l-1]*p[r-l+1];
22 }
23
24 int lcp(int x,int y){//二分查找后缀x和后缀y的最长公共前缀
25     int l = 0,r = min(n-x+1,n-y+1);
26     while(l < r){
27         int mid = l + r + 1 >> 1;
28         if(query(x,x+mid-1) == query(y,y+mid-1)) l = mid;
29         else r = mid - 1;
30     }
31     return l;
32 }
33
34 bool cmp(int x,int y){
35     int len = lcp(x,y);
36     return s[x+len] < s[y+len];//s[x+len]和s[y+len]即为第一个不同的位置
37 }
38
39 int main(){
40     cin >> s; n = s.size();
41     init(s);
42     s = ' ' + s;
43
44     for(int i = 1;i <= n;i++) sa[i] = i;
45
46     stable_sort(sa+1,sa+n+1,cmp);
47
48     for(int i = 1;i <= n;i++){
49         cout << sa[i] - 1 << ' ';    //sa[] 本题下标从0开始,减1即可
50         rk[sa[i]] = i;//rk[]
51     }
52     cout << '\n';
53     for(int i = 1;i <= n;i++){
54         cout << lcp(sa[i],sa[i-1]) << ' ';//height[]
55     }
56 }

```

倍增实现

时间复杂度 $O(N\log N)$

```
1 //https://www.luogu.com.cn/problem/P10469
2 #include <bits/stdc++.h>
3
4 namespace SA{//1_idx 未验证多测是否正确初始化
5     int n;
6     std::vector<int>sa,c,x,y,rk,height;
7
8     template<typename T>
9     void build (const T &s) {//get:sa[]
10         sa = x = y = std::vector<int>(n+1);
11         int m = 300;//字符值域
12         c.resize(std::max(n,m)+1);
13         for (int i = 1; i <= m; i++) c[i] = 0;
14         for (int i = 1; i <= n; i++) c[x[i] = s[i]]++;
15         for (int i = 1; i <= m; i++) c[i] += c[i-1];
16         for (int i = n; i >= 1; i--) sa[c[x[i]]--] = i;
17         for (int j = 1; j <= n; j <= 1) {
18             int p = 0;
19             for (int i = n - j + 1; i <= n; i++) y[++p] = i;
20             for (int i = 1; i <= n; i++) if (sa[i] > j) y[++p] = sa[i] - j;
21             for (int i = 1; i <= m; i++) c[i] = 0;
22             for (int i = 1; i <= n; i++) c[x[y[i]]]++;
23             for (int i = 1; i <= m; i++) c[i] += c[i-1];
24             for (int i = n; i >= 1; i--) sa[c[x[y[i]]]--] = y[i];
25             std::swap (x, y);
26             p = 1;
27             x[sa[1]] = 1;
28             for (int i = 2; i <= n; i++) {
29                 x[sa[i]] = y[sa[i-1]] == y[sa[i]] && y[sa[i-1]+j] == y[sa[i]+j]
30                 ? p : ++p;
31             }
32             if (p >= n) break;
33             m = p;
34         }
35
36         template<typename T>
37         void make (const T &s) {//get:rk[],height[]
38             rk = height = std::vector<int>(n+1);
39             int k = 0;
40             for (int i = 1; i <= n; i++) rk[sa[i]] = i;
41             for (int i = 1; i <= n; i++) {
42                 if (rk[i] == 1) continue;
43                 if (k) k--;
44                 int j = sa[rk[i] - 1];
45                 while (j + k <= n and i + k <= n and s[i + k] == s[j + k]) {
```

```

46         ++ k;
47     }
48     height[rk[i]] = k;
49 }
50 }
51
52 template<typename T>
53 std::vector<int> get_sa(const T &s){
54     n = s.size()-1;
55     build(s);
56     make(s);
57     return sa;
58 }
59 }
60 using SA::get_sa, SA::sa, SA::rk, SA::height;
61
62
63 int main () {
64     std::string s; std::cin >> s; s = ' ' + s;
65     int n = s.size()-1;
66
67     get_sa(s);
68
69     for(int i = 1; i <= n; i++){ std::cout << sa[i]-1 << ' '; }
70     std::cout << '\n';
71     for(int i = 1; i <= n; i++){ std::cout << height[i] << ' '; }
72 }

```

height数组

lcp 为最长公共前缀，下文以 $lcp(i, j)$ 表示后缀 i 和后缀 j 的最长公共前缀。

`height[ ]` 数组的定义：`height[i] = lcp(sa[i], sa[i-1])`，即第 i 名的后缀与它前一名后缀的最长公共前缀。`height[1]` 可以视作 0。

一些应用

两子串最长公共前缀

`lcp(sa[i], sa[j]) = min({height[i+1...j]})`

如果 `height` 一直大于某个数，前这么多位就一直没变过；反之，由于后缀已经排好序了，不可能之后变回来。

求两子串的最长公共前缀就转化为了RMQ问题。

比较两子串的大小关系

假设比较的子串为 $A = s[a, b]$ 和 $B = s[c, d]$

若 $lcp(a, c) \geq \min(|A|, |B|)$, $A < B \iff |A| < |B|$ 。
否则, $A < B \iff rk[a] < rk[c]$ 。

求字符串循环同构的字典序排名

[P4051 [JSOI2007 字符加密](#) - 洛谷 (luogu.com.cn)]

将s复制一遍后, 求sa[]

求不同子串的数目

[P2408 不同子串个数](#) - 洛谷 (luogu.com.cn)

每个子串一定是某个后缀的前缀, 所以可以计算子串总数, 枚举每个后缀, 减掉重复。

$$\text{答案为: } \frac{n(n+1)}{2} - \sum_{i=2}^n height[i]$$

求至少出现k次的子串(可重叠)的最大长度

[P2852 [USACO06DEC Milk Patterns G](#) - 洛谷 (luogu.com.cn)]

出现至少k次意味着后排序后, 有至少连续k个后缀以这个子串作为公共前缀。

所以求出每相邻k-1个height的最小值, 这些最小值的最大值即为答案, 这里可以用单调队列O(n)解决

结合并查集/线段树/单调栈等数据结构

其它

求字符串S中长度为k的子序列中, 字典序最小的一个。 $1 \leq K \leq N \leq 10^5$

贪心实现, 依次考虑子序列的每一位, 从合法区间[l,r]中选择字典序最小的一个字母, 选择该字母后, 该字母前面的字母都不能选, 于是我们可以不断缩小合法区间。

```
1 //https://vjudge.net/problem/AtCoder-typical90_f#author=GPT_zh
2 #include <iostream>
3 #include <queue>
4 using namespace std;
5 int n,k;
6 string s;
7 queue<int>q[30];
8
9 int main(){
10     cin >> n >> k >> s;
11     for(int i = 0; i < s.size(); i++){
12         q[s[i]-'a'].emplace(i);
13     }
14
15     int l = 0;
16     string ans;
17     for(int i = 0; i < k; i++){
18         int r = n - k + i; //右端点必须小于r, 否则凑不齐长度k的序列
19         for(int c = 0; c < 26; c++){
```



```

20         while(q[c].size() && q[c].front() < l){ //左端点l前面的字母都可以排除
21             q[c].pop();
22         }
23         if(q[c].empty()) continue;
24         if(q[c].front() <= r){ //选择该位后,缩小左端点
25             ans += 'a' + c;
26             l = q[c].front();
27             q[c].pop();
28             break;
29         }
30     }
31 }
32 cout << ans;
33 }

```

给定一个长度为N的字符串S，求有多少个子序列 = 字符串T。

设dp[i]为T的前i个前缀出现的次数，时间复杂度 $O(NM)$

```

1 //https://vjudge.net/problem/AtCoder-typical90_h#author=GPT_zh
2 #include <bits/stdc++.h>
3 using namespace std;
4 const int N = 200005, mod = 1e9+7;
5 string t = "atcoder";
6 int a[N];
7 long long dp[5];
8
9 int main(){
10     int n; cin >> n;
11     int m = t.size()-1;
12     string s; cin >> s; s = ' ' + s;
13
14     dp[0] = 1;
15     for(int i = 1; i <= n; i++){
16         for(int j = 1; j <= m; j++){
17             if(s[i] == t[j]){
18                 dp[j] = (dp[j] + dp[j-1])%mod;
19             }
20         }
21     }
22     cout << dp[m] << '\n';
23 }

```